

# Re-Pricing the iGPU After Periodicity: The $-23\%$ Win Collapses to a Wash

Matias Saavedra

Wednesday 8<sup>th</sup> July, 2026

Binary: merge commit 833a791 on branch feat/igpu-opencl (OpenCL third executor, 2991dd9)  $\times$  main (c4744c0, code fa6d8ac = tag binary-v6-periodicity). Probe 2 adds one line to kernel.cu (commit 29c68ea). Raw data: experiments/22-igpu-atop-v6/.

## Resumen

We re-measure the integrated-GPU backend of 20-igpu-opencl.tex on a binary that also carries the periodicity check of 21-periodicity-check.tex. Method: the same production A/B (100 frames,  $1920 \times 1080$ , diffT = 0.1, save=1, three alternated reps), dGPU+iGPU+10CPU vs dGPU+11CPU. **The iGPU's  $-23.2\%$  collapses to a wash:**  $28.05 \pm 0.12\text{ s}$  vs  $27.36 \pm 1.60\text{ s}$  ( $+2.5\%$  on all-reps means,  $-0.7\%$  excluding a cold first run). The mechanism is an exact cancellation: the iGPU strips 27 CPU-seconds off the binding CPU pool, and the CPU worker it displaces would have contributed 26.5. Expressed as marginal value, periodicity re-priced the dGPU from  $\sim 23$  to 2.7 CPU-worker equivalents and the iGPU from  $\sim 13$  to  $\approx 1$ . Two follow-up probes kill the obvious rescue: 13-thread oversubscription loses  $+7.4\%$  (the lanes busy-spin), and with cudaDeviceScheduleBlockingSync the recovered worker lands past the SMT knee — a wash that also closes the report-15 confirmation item. Recommendation: keep feat/igpu-opencl off main.

## 1. Background and question

Report 20-igpu-opencl.tex added a third executor — an OpenCL FP64 backend on the Intel UHD 630 — and measured a surprise: replacing one CPU worker with the iGPU lane cut the production wall from 57.72 to 44.32 s ( $-23.2\%$ ). The mechanism was a second *serial lane* for the big coherent interior regions, stripping 187 s of MAXITER-grind off the binding CPU pool.

Report 21-periodicity-check.tex then attacked that same grind directly: an exact Brent cycle detector in the CPU `diverge()` lets an interior pixel return after  $\sim 900$  iterations-equivalent instead of 10,000, byte-identically. The hybrid wall fell  $57.6 \rightarrow 30.9\text{ s}$  and its closing section flagged the consequence measured here: *the iGPU's  $-23.2\%$  was priced against work that no longer exists*. This experiment answers the re-opened question — and, having both changes in one binary for the first time, prices every device in the system against the new, faster CPU worker.

## 2. Methodology

## Binary.

`main` (carrying `binary-v6-periodicity`) merged into `feat/igpu-openc1` at commit 833a791; the two changes touch disjoint functions (`diverge()` vs the `ExecKind` dispatch in `compute()`) and auto-merged. One binary, both changes; all runs in this report use it. Probe 2 rebuilds with one added line ( $\mathcal{S}4.4$ ).

## Workload and configurations.

Production `spec.in` (100 frames,  $1920 \times 1080$ , deep zoom to `maxIter` 10,000), `diffT` = 0.1, `pixT` = 32,768, `quiet`=1, `save`=1 — the same Fig.-11.15 methodology as the experiment-20 and 21 headlines. Headline arms: mode 1 (`dGPU+11CPU`) vs mode 3 (`dGPU+iGPU+10CPU`), 12 threads total, three reps with the two modes alternated inside each rep so thermal drift hits both equally. Hardware: i7-9750H (6C/12T) + GTX 1660 Ti Max-Q + UHD 630 (`intel-compute-runtime-legacy1`), on AC power.

## Calibration honesty.

Mode 1 here runs  $\sim 9\%$  faster than experiment 21’s identical configuration measured one day earlier (27.4–28.3s steady-state vs  $30.94 \pm 0.56$ ). The difference is thermal history: experiment 21 interleaved 163-second CPU12 baseline grinds before every hybrid run; this sweep is six  $\sim 28$ s runs. Within-batch, the first (coldest) run of the day is reproducibly the fastest by 1.5–3s. Absolute walls on this laptop therefore carry a  $\sim 10\%$  thermal-history band; every comparison in this report is same-batch and alternated, which the band does not affect.

## 3. Results

Tabla 1: Headline A/B (3 alternated reps, `save`=1). The pre-v6 rows reproduce experiment 20 on the binary-v5 lineage for context; post-v6 rows are this experiment. Read  $\Delta$  as the marginal effect of swapping the 11th CPU worker for the iGPU lane: a  $-13.4$ s win before periodicity, a wash after.

| binary  | config                              | mean (s)     | $\sigma$ | $\Delta$ vs mode 1 |
|---------|-------------------------------------|--------------|----------|--------------------|
| pre-v6  | mode 1 <code>dGPU+11CPU</code>      | 57.72        | 0.35     | —                  |
| pre-v6  | mode 3 <code>dGPU+iGPU+10CPU</code> | <b>44.32</b> | 0.26     | $-23.2\%$          |
| post-v6 | mode 1 <code>dGPU+11CPU</code>      | <b>27.36</b> | 1.60     | —                  |
| post-v6 | mode 3 <code>dGPU+iGPU+10CPU</code> | 28.05        | 0.12     | $+2.5\%$           |

The post-v6 delta is  $+0.69\text{s} = 0.43\sigma$  of the mode-1 spread — not a real slowdown, but decisively not the  $-13.4$ s of experiment 20. Mode 1’s  $\sigma$  is inflated by its first rep (25.54s, the cold first run of the batch); excluding it, mode 1 = 28.27s and the delta is  $-0.7\%$ . Either reading is a wash. All six runs produce the identical 5,608-leaf decomposition.

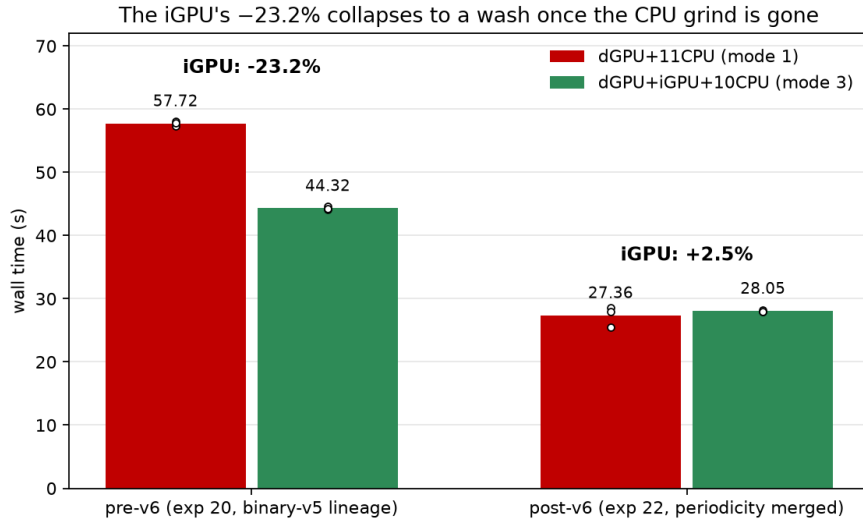


Figure 1: Wall time by era and configuration; white dots are individual reps. Red is mode 1 (dGPU+11CPU), green is mode 3 (dGPU+iGPU+10CPU). The left pair is experiment 20’s -23.2%; the right pair is the same A/B once periodicity is merged in: the bars swap order and the gap falls inside run-to-run noise.

Tabla 2: Executor split and pool accounting, steady-state reps (r2/r3 means). “Pool busy” sums the per-thread **Total compute time** counters. The load-bearing column is the last: the per-worker load is identical, so removing a worker exactly consumes what the iGPU contributes.

| config          | pool regions | dGPU  | iGPU | pool busy (s) | s/thread    |
|-----------------|--------------|-------|------|---------------|-------------|
| mode 1 (11 CPU) | 4,662        | 946   | —    | 292           | <b>26.6</b> |
| mode 3 (10 CPU) | 2,861        | 1,761 | 986  | 266           | <b>26.6</b> |

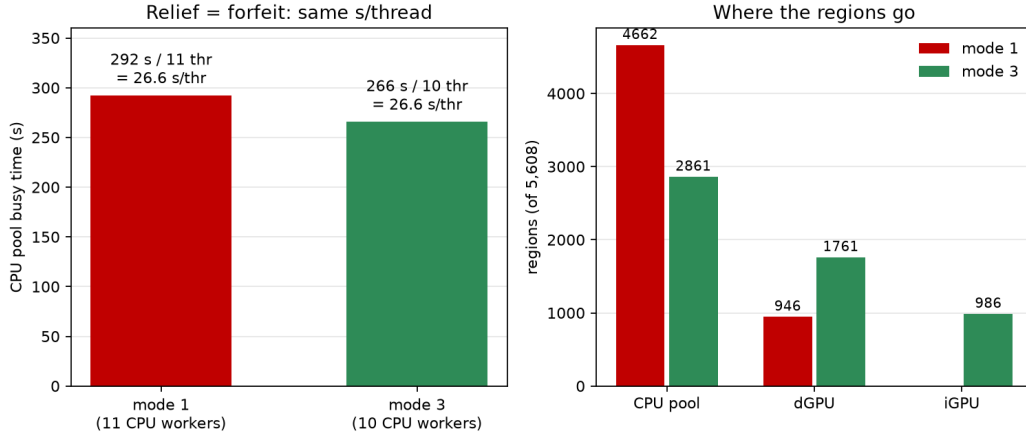


Figure 2: The wash as arithmetic. Left: total CPU-pool busy time — the iGPU relieves 27s of pool work (292→266s), but spread over one fewer worker the per-thread load is unchanged at 26.6s, and the busiest CPU thread ( $\approx$  wall) is the same in both modes. Right: where the 5,608 regions execute — in mode 3 the two GPUs absorb 2,747 regions, yet the pool’s remaining 2,861 still bind.

## 4. Analysis

### 4.1. The swap arithmetic: relief equals forfeit

Table 2 is the whole result. The wall in both modes is set by the CPU pool (busiest thread  $\approx$  wall in all six runs). Swapping the 11th worker for the iGPU lane removes  $292 - 266 = 27$  CPU-seconds from the pool; the worker it displaced would have supplied  $292/11 = 26.5$  CPU-seconds of capacity in the same wall time. The two sides of the trade agree to within 2%, which is exactly what a wash looks like from inside.

The reason is *what* the iGPU takes. GPU affinity (report `12-gpu-affinity.tex`) hands both GPU lanes the largest queued regions, which at `diffT=0.1` are certified-interior content — precisely the region class where the periodicity check makes a CPU worker  $\sim 11\times$  faster (the pure-interior single-region test of experiment 21:  $67.27 \rightarrow 5.98$ s). The pool work that remains — the truly-escaping boundary band plus detection latency — lives in small pixel-floor regions the GPUs never pull. The iGPU is relieving the pool of the one thing the pool is no longer slow at.

Contrast experiment 20: the same swap then stripped 187s off a 620s pool, because interior grind at 10,000 iterations per pixel was the CPU’s pathology. That work class is gone; the win went with it.

### 4.2. Periodicity re-priced every accelerator, not just the iGPU

The marginal value of a device can be expressed in CPU-worker equivalents  $k$  from the config walls, using the makespan model  $\text{wall} \propto W/(\text{effective workers})$  — valid here be-

cause the pool binds and the decomposition (hence  $W$ ) is identical across configs. Replacing one of 12 CPU workers with a lane of value  $k$  gives  $k = 12(\text{wall}_{\text{CPU}12}/\text{wall}_{\text{lane}}) - 11$ .

Tabla 3: Marginal device value in CPU-worker equivalents, before and after periodicity. Sources: pre-v6 dGPU from experiment 21’s matched baseline pair (164.43/57.64 s); pre-v6 iGPU from experiment 20’s mode 2 (161.64/81.72 s); post-v6 dGPU from experiment 21 (35.21/30.94 s); post-v6 iGPU from this experiment’s wash, solved from the mode-1/mode-3 ratio ( $k_i \approx 0.7$ –1.1 depending on whether the cold rep is included). Both devices collapse by roughly the interior iteration-elimination ratio.

| device                         | pre-v6 $k$ | post-v6 $k$   | collapse           |
|--------------------------------|------------|---------------|--------------------|
| dGPU (GTX 1660 Ti Max-Q, CUDA) | 23.2       | 2.7           | $\nabla \cdot 8.7$ |
| iGPU (UHD 630, OpenCL)         | 12.7       | $\approx 1.0$ | $\nabla \cdot 13$  |

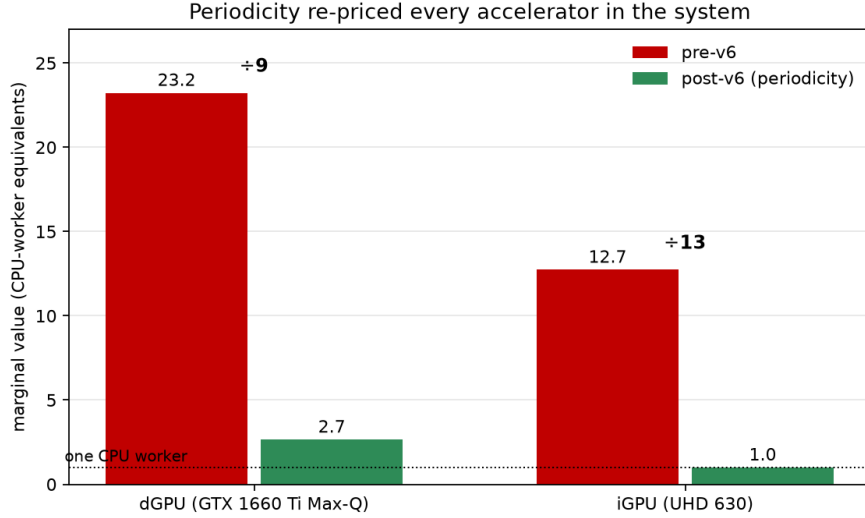


Figure 3: Marginal device value in CPU-worker equivalents (red pre-v6, green post-v6); the dotted line is one CPU worker. The iGPU lands exactly on the line — which is the geometric statement of the wash: swapping it for a worker changes nothing.

The collapse factors ( $8.7\times$  and  $13\times$ ) bracket the pure-interior elimination ratio ( $11.2\times$ ), and that is not a coincidence: the GPUs’ diet is interior-dominated by construction (affinity feeds them the certified-uniform largest regions), so their worker-equivalence scales down by almost exactly the discount the CPU received on that content. The important framing: **the iGPU did not get slower**. Its raw iteration throughput is unchanged — of order 15–20 scalar CPU threads. Its *marginal* value collapsed because report 21 removed 10/11 of the iterations a CPU spends on the content the iGPU serves, while both GPU kernels still grind all 10,000 (deliberately: report 16-ncu-divergence.tex showed the interior kernels are 100% warp-coherent and FP64-bound, so an early-out would introduce divergence). The dGPU survives

at  $k = 2.7$  because it is  $\sim 1.7\times$  faster per region than the iGPU, runs its lane nearer saturation, and does not share the CPU package’s power budget.

### 4.3. Probe 1: keeping all eleven workers by oversubscribing loses

If the wash is just “the lane costs a worker,” the obvious rescue is to pay with a thread the hardware does not have: 13 threads = 11 CPU workers + both lanes on 12 logical CPUs.

Tabla 4: Probe 1 (2 alternated reps each, same batch). Oversubscription is strictly negative: worse than both 12-thread configurations.

| config                   | reps (s)     | mean          |
|--------------------------|--------------|---------------|
| mode 1, 12 thr (control) | 26.28, 28.43 | <b>27.36</b>  |
| mode 3, 13 thr           | 29.58, 29.18 | 29.38 (+7.4%) |

The reason is that the premise “the GPU thread is idle while the kernel runs” is false on this system. Report `15-async-streams-analysis.tex` established that the CUDA synchronous wait *busy-spins* (driver `ScheduleAuto` spins when contexts < cores), and both lanes additionally execute the host-side commit (the per-pixel `set-Pixel` loop) — so each lane genuinely occupies a logical CPU. Thirteen runnable threads on twelve logical CPUs just timeshare: the pool’s busiest thread rises 26.5–27.9 → 28.2–29.0s and the pool’s busy-time counters inflate 265 → 305s (the per-thread timers are wall-clock, so descheduled time counts). The 11th worker’s nominal 27s of capacity is consumed by the contention it creates.

### 4.4. Probe 2: BlockingSync — the recovered worker is past the SMT knee

Probe 1’s failure is the spin’s fault, so probe 2 removes the spin — the one-line confirmation report 15 proposed:

Código 1: kernel.cu, top of `CUDAmemSetup()` (commit `29c68ea`) – the driver now sleeps the dGPU lane on synchronous waits.

```

1 // Make the driver sleep the calling thread on synchronous waits instead of
2 // busy-spinning (default ScheduleAuto spins when contexts < cores). Must be
3 // set before the first context-creating runtime call below.
4 CUDA_CHECK_RETURN(cudaSetDeviceFlags(cudaDeviceScheduleBlockingSync));

```

Tabla 5: Probe 2 (2 alternated reps each, one batch). Read each row against the 12-thread control in the first: the freed worker (row 2) is a wash, and mode 3 (row 3) improves but still never beats plain mode 1.

| config                                  | reps (s)     | mean         | pre-BlockingSync |
|-----------------------------------------|--------------|--------------|------------------|
| mode 1, 12 thr (control)                | 24.33, 27.04 | <b>25.69</b> | 27.36            |
| mode 1, 13 thr (12 CPU + sleeping lane) | 26.30, 26.52 | 26.41        | —                |
| mode 3, 13 thr (11 CPU + both lanes)    | 28.63, 26.87 | 27.75        | 29.38            |

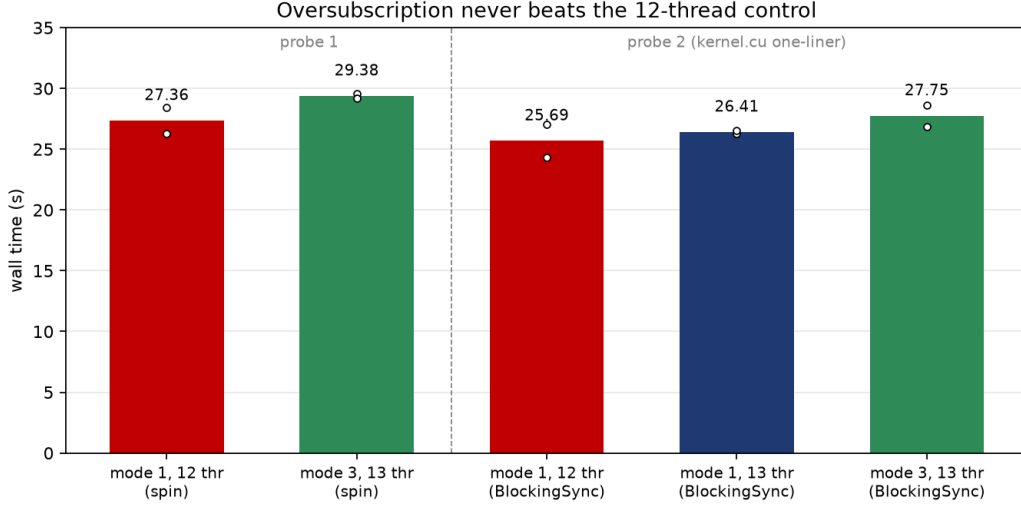


Figura 4: The probe ladder; white dots are reps. Left of the dashed line: busy-spinning lanes make 13 threads strictly worse. Right: with the dGPU lane sleeping, the recovered 12th worker (blue) is a wash against the control (red), and three devices (green) still lose to two. No arm beats the plain 12-thread mode 1.

Three readings, in order of confidence:

### BlockingSync is safe and roughly neutral.

Every configuration shifted  $\approx -1.6$ s against the pre-BlockingSync batch, but the first-of-batch cold effect is the same magnitude (24.33s here, 25.54s in the morning batch), so we do not claim the shift. What the data does establish: no regression — sleep/wake latency is invisible at 20–70 ms/region. This closes report 15’s “cheap confirmation” item; the line stays.

### The recovered worker adds only past-the-knee capacity.

With the lane sleeping, the 12th worker genuinely ran (pool busy 296s vs 263–276s absorbed) — but per-region CPU cost rose 53  $\rightarrow$  57ms and the wall did not move (26.41 vs 25.69, overlapping spreads). The 12th worker lands on the SMT sibling of an already-busy core, worth the  $\sim 0.3$ – $0.4$  physical cores report 15 priced for exactly this position — while the busy-spinner it displaced was already SMT-polite (a **PAUSE** loop yields core resources to its sibling). The naive model “freeing a spinning logical

CPU recovers 1/11 of the pool” double-counts what the spin never really took.

### Mode 3 loses under every scheme tried.

Worker swap (+2.5%), spin oversubscription (+7.4%), blocking oversubscription (+8.0% vs its own control). The CUDA flag does not touch the OpenCL lane’s wait, both lanes still carry their host-side commits, and the iGPU shares the 45 W package budget with the pool it is supposed to be helping. This also closes the “one thread drives both GPUs” multiplexer idea without writing it: its entire prize is the recovered worker just measured at noise level.

## 4.5. Limits

The headline A/B is 3 reps; the probes are 2. Within-config scatter reaches 2.7s (cold-start), so the probes can rule out the  $-8\%$ -class effects they hunted but cannot rank the  $\sim 1$ s differences among surviving configs. The  $k$  model of  $\mathcal{S}4.2$  assumes a binding, work-conserving pool; it is a characterization tool, and its post-v6 iGPU value is bounded by the same noise as the wash it is derived from ( $k_i \in [0.7, 1.1]$ ). Mode 2 (iGPU+11CPU) was not re-run post-v6, and the OpenCL lane’s wait-vs-spin behaviour on the NEO runtime was not investigated — neither affects the merge decision, since both could only matter in configurations that already lose.

## 5. Recommendations

1. **Do not merge feat/igpu-opengl into main.** Measured value atop v6:  $+2.5\%/-0.7\%$  (a wash) for the cost of an OpenCL ICD loader dependency at build time, the AUR legacy Intel runtime at run time, and a third executor in the code. The branch stays as a characterization — report 20 documents the pre-v6 win, this report its repeal — mirroring the reports-11/18 precedent.
2. **Keep the cudaDeviceScheduleBlockingSync line** (it stays on this branch; porting to **main** is optional and expected to be worth  $\approx 0$  at 12 threads). It is measured safe, and it retires report 15’s open confirmation.
3. **The one lever left on this axis is GPU-side periodicity** (report 21, recommendation 3).  $\mathcal{S}4.2$  sharpens its motivation: the GPUs’ value collapsed because only the CPU got the  $\sim 11\times$  interior discount. A GPU-side check would re-inflate  $k$  for both GPUs — against report 16’s warning that it breaks the kernels’ 100% warp coherence. That trade must be measured, not assumed, and it is a separate A/B on **kernel.cu**.
4. **If the iGPU is ever revisited** (e.g. after GPU-side periodicity changes the price again): the unmeasured residuals are the **scanLine** commit cheapening and strict dGPU-priority-on-largest (report 20  $\mathcal{S}$ Analysis), plus mode 2 as a budget config for machines without a dGPU.
5. **Treat absolute walls on this laptop as carrying a  $\sim 10\%$  thermal-history band.** Same-day alternated controls (as used throughout here) are mandatory for any future headline number; cross-day comparisons of raw seconds are not meaningful at the current effect sizes.

## 6. Conclusion

Experiment 22 answers the question report 21 re-opened: with the periodicity check merged, the integrated GPU's  $-23.2\%$  becomes  $+2.5\%/-0.7\%$  — a wash, mechanism fully accounted (27 CPU-seconds relieved vs 26.5 forfeited, per-worker load unchanged at 26.6s/thread). Two probes eliminated the rescue paths: oversubscription loses to the busy-spinning lanes ( $+7.4\%$ ), and with BlockingSync the recovered worker is worth only past-the-SMT-knee capacity (wash), which also retires the dual-GPU multiplexer idea and closes report 15's confirmation item. The general lesson is the repricing table: work-reduction on one executor devalues every other executor whose niche was that work — the dGPU fell from  $\sim 23$  to 2.7 worker-equivalents, the iGPU from  $\sim 13$  to  $\approx 1$ , both by roughly the interior iteration-elimination ratio. **feat/igpu-openc1** should remain unmerged; the next lever on this axis, if any, is giving the GPUs the same discount the CPUs got.